Assignment 4 – UML Use Case Diagram: Sudoku Solver

Francis Tryban · EECS 348 · Fall 2026 · Made with help from Claude Opus 5.5 (Anthropic), a GenAI that is not one of the two models compared in my GenAI analysis.

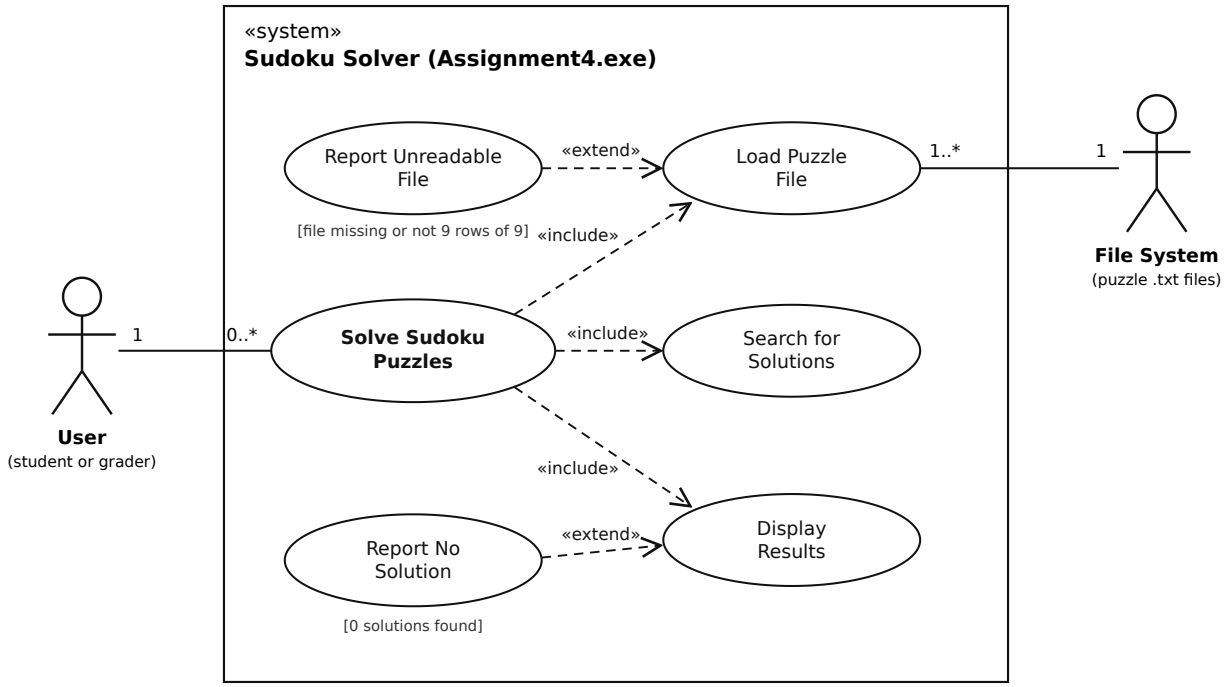


Figure 1. Use case diagram for the Sudoku Solver. «include» arrows point from the base use case to the part it always uses. «extend» arrows point from the optional behavior to the use case it adds to, with the condition shown in brackets.

a. Identify / discover the use case

System: the Sudoku Solver program (`Assignment4.exe`), run from a terminal on the cycle server.

Actors:

- **User** (primary actor): the person who runs the program. This is a role, so a student testing it and a grader running it are both the User.
- **File System** (secondary actor): outside the program. It holds the puzzle text files the program opens and reads.

Main use case discovered: Solve Sudoku Puzzles. Written as a goal: *As a User, I want to run the solver on my puzzle files so that I can see every solution to each puzzle, or know that a puzzle has none.*

Use cases factored out of it:

- **Load Puzzle File** («include»): open one file and read its 9x9 grid. Always needed.
- **Search for Solutions** («include»): the recursive depth-first search with backtracking. Always needed.

- **Display Results** («include»): print the file name, the puzzle as read, each solution, and the count. Always needed.
- **Report Unreadable File** («extend» on Load Puzzle File): only happens when a file is missing or is not 9 rows of 9 cells.
- **Report No Solution** («extend» on Display Results): only happens when the search finds 0 solutions.

Multiplicities:

- User **1** — Solve Sudoku Puzzles **0..***: exactly one person runs each solve, and one person can run the program as many times as they want (zero or more).
- File System **1** — Load Puzzle File **1..***: there is one file system, and every run loads at least one puzzle file (the five course files by default, or the files typed on the command line).

b. Brief description

This use case lets a User solve one or more Sudoku puzzles stored in text files. For each file, the system reads the puzzle, prints it, and uses a recursive brute-force search with backtracking to find every way to fill the empty cells without breaking a Sudoku rule. It prints each solution as it is found and then says how many there were. If a puzzle has no solution, it prints "No solution found". If a file cannot be read, it says so and moves on to the next file.

c. Step-by-step flow (outline)

1. Run the program
2. Choose the puzzle files
3. Open the next puzzle file
4. Read the 9x9 grid
5. Check the clues for rule breaks
6. Display the file name and the puzzle
7. Search for solutions
8. Display each solution as it is found
9. Display the number of solutions (or "No solution found")
10. Repeat steps 3–9 for each remaining file, then end

d. Detailed description

Use case	Solve Sudoku Puzzles
Primary actor	User
Secondary actor	File System
Preconditions	The program has been compiled on cycle (`g++ Assignment4.cpp -o Assignment4.exe`). The puzzle files are in the same folder, or their names are typed after the program name. Each puzzle is 9 rows of 9 cells, with a digit 1–9 for a clue and _ for a blank, separated by spaces.
Postconditions	Every chosen file has been reported once: its puzzle and all of its solutions with a count, "No solution found", or "Unable to read puzzle". The puzzle files are not changed.

Basic flow

1. User runs the program

The use case starts when the User types `./Assignment4.exe` in the folder with the puzzle files, with or without file names after it.

2. System chooses the puzzle files

If the User typed file names, the system makes a list of those names in the order typed. If not, the system uses `puzzle1.txt` through `puzzle5.txt`.

3. System opens the next puzzle file (Load Puzzle File)

The system prints the file name and asks the File System to open that file. The File System hands back the file's contents. *Extension point: if the file cannot be opened, see A1.*

4. System reads the 9x9 grid (Load Puzzle File)

The system reads the file line by line. In each line it keeps the digits 1–9 as clues and each _ as a blank, and skips spaces and Windows line endings. A line with 9 cells becomes the next row. Blank lines are skipped. Reading stops after 9 rows. *Extension point: if a line has some cells but not 9, or the file has fewer than 9 rows, see A1.*

5. System checks the clues

For each clue, the system checks whether the same digit is already in its row, its column, or its 3x3 box. If two clues break a rule, the system marks the puzzle as having no solution (see A2).

6. System displays the puzzle (Display Results)

The system prints the grid exactly as read, 9 rows of 9 cells separated by spaces, with _ for blanks, so the User can check the puzzle was read correctly.

7. System searches for solutions (Search for Solutions)

The system finds the first blank cell (left to right, top to bottom) and tries the digits 1 to 9 in it. For each digit that does not break a rule, it places the digit and calls the same search again for the next blank cell. When a path hits a dead end, it erases the digit and tries the next one (backtracking). When no blank cells are left, the grid is a complete

solution. The search does not stop at the first solution. It keeps going until every possibility has been tried.

8. System displays each solution as it is found (Display Results)

Each time the search fills the whole grid, the system prints "Solution N:" and the finished grid right away, then the search continues. Solutions are not stored, so memory use stays the same no matter how many solutions there are.

9. System displays the number of solutions (Display Results)

When the search is done, the system prints "Solutions found: N". *Extension point: if N is 0, see A3.*

10. System moves on, then ends

The system prints a blank line and repeats steps 3–9 for the next file in the list. After the last file, the program ends and returns 0 to the terminal. The use case ends.

Alternate flows

A1. Report Unreadable File (extends Load Puzzle File)

If the file is missing, cannot be opened, has a row that is not 9 cells, or has fewer than 9 rows, the system prints "Unable to read puzzle" under the file name. It does not search that file and goes back to step 3 for the next file.

A2. Clues already break a rule

If step 5 found two clues that conflict, the system still prints the puzzle (step 6) but skips the search (steps 7–8), because no solution can exist. It continues at A3.

A3. Report No Solution (extends Display Results)

If the search found 0 solutions, the system prints "No solution found" instead of a count, then continues at step 10. (Example: `puzzle4.txt`.)

A4. Puzzle with more than one solution

Nothing special happens. Steps 7–8 print every solution, numbered, and step 9 prints the total. (Example: `puzzle5.txt` prints Solution 1 and Solution 2, then "Solutions found: 2".)

GenAI used: Claude Opus 5.5 (Anthropic) helped make this diagram and description from my program and the "UML Use Case Modeling" lecture slides. I checked that it covers the four levels from the lecture (identified, briefly described, outlined, fully described) and that the steps match what my program actually does.